

Questão 01 - Dockerfile para o Lift

Prompt (aplicando R-T-F)

[ROLE]

Você é um Engenheiro DevOps Sênior, especialista em containerização de aplicações Python com Docker e em boas práticas de deploy para clusters Kubernetes.

[TASK]

Crie um Dockerfile de produção para a aplicação "Lift", uma API Python/Flask localizada na raiz do projeto (app.py), com dependências declaradas em requirements.txt e código auxiliar no diretório lib/ (auth.py, storage.py). O diretório tests/ não deve ser incluído na imagem final.

Requisitos obrigatórios:

- A aplicação deve subir via: `gunicorn --bind 0.0.0.0:8080 --workers 4 app:app`
- A porta exposta deve ser 8080
- As variáveis de ambiente `DATABASE_URL` e `API_KEY` são obrigatórias em runtime, mas NÃO devem ter valores fixos ou secretos embutidos na imagem — apenas documentadas/declaradas para injeção via Kubernetes (Secret/ConfigMap)
- Aplique boas práticas de Docker: build multi-stage, imagem base slim, usuário non-root, cache eficiente de camadas, `.dockerignore` recomendado, minimização de camadas e do tamanho final da imagem

[FORMAT]

Entregue a resposta em duas partes:

1. O conteúdo completo do Dockerfile em um bloco de código, com comentários explicando o propósito de cada instrução relevante
2. Uma lista curta (bullet points) com o conteúdo sugerido para o `.dockerignore`

Não inclua explicações fora dessas duas partes.

Modelo de IA utilizado

Claude Sonnet 4.5 (Anthropic)

Output gerado

```
# ----- Build stage -----
```

```
FROM python:3.12-slim AS builder
```

```
WORKDIR /app
```

```
# Copia apenas o requirements primeiro para aproveitar cache de camadas do Docker
```

```
COPY requirements.txt .
```

```
# Instala dependências em diretório de usuário, sem cache de pip (imagem menor)
```

```
RUN pip install --no-cache-dir --user -r requirements.txt
```

```
# ----- Runtime stage -----
```

```
FROM python:3.12-slim
```

```
# Cria usuário e grupo non-root por segurança (evita rodar app como root)
```

```
RUN groupadd -r lift && useradd -r -g lift lift
```

```
WORKDIR /app
```

```
# Copia apenas as dependências já instaladas do stage de build
```

```
COPY --from=builder /root/.local /home/lift/.local
```

```
# Copia somente o código necessário para rodar a aplicação (tests/ fica de fora)
```

```
COPY app.py .
```

```
COPY lib/ ./lib/
```

```
ENV PATH=/home/lift/.local/bin:$PATH \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
```

Variáveis obrigatórias em runtime — SEM valores fixos.

Devem ser injetadas via Secret/ConfigMap do Kubernetes no deploy.

```
ENV DATABASE_URL="" \
    API_KEY=""
```

Garante que os arquivos copiados pertencem ao usuário non-root

```
RUN chown -R lift:lift /app /home/lift/.local
```

```
USER lift
```

```
EXPOSE 8080
```

Healthcheck simples para o orquestrador validar se o container está saudável

```
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
```

```
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')"
```

```
    || exit 1
```

```
CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "4", "app:app"]
```

.dockerignore sugerido:

- tests/
- .git
- .env
- __pycache__/
- *.pyc
- .pytest_cache
- README.md

Justificativa — como R-T-F aparece no prompt

Componente	Onde aparece	Por quê
Role	"Engenheiro DevOps Sênior, especialista em containerização... e Kubernetes"	Ancora a IA numa persona técnica específica, o que direciona o vocabulário e o nível de rigor das boas práticas aplicadas (multi-stage, non-root, etc.) — sem isso, a IA poderia entregar um Dockerfile genérico de tutorial
Task	Bloco de requisitos: estrutura do projeto, comando gunicorn, porta, variáveis de ambiente, boas práticas exigidas	Define exatamente o que precisa ser produzido e as restrições (não hardcodar secrets, excluir tests/, seguir o comando de produção informado no enunciado)
Format	Especificação de "duas partes: Dockerfile + lista do .dockerignore", com comentários no código e sem explicações extras	Garante que o output seja direto e utilizável (copiar e colar), no formato que um DevOps esperaria receber, sem texto solto desnecessário